EECS 182 Deep Neural Networks
Spring 2025 Anant Sahai

Homework 6

This homework is due on March 16, at 10:59PM.

1. Graph Dynamics

Some graph neural network methods operate on the full adjacency matrix. Others, such as those discussed here https://distill.pub/2021/gnn-intro/, at each layer apply the same local operation to each node based on inputs from its neighbors.

This problem is designed to:

- Show connections between these methods.
- Show that for a positive integer k, the matrix A^k has an interesting interpretation. That is, the entry in row i and column j gives the number of walks of length k (i.e., a collection of k edges) leading from vertex i to vertex j.

To do this, let's consider a very simple deep linear network, defined as follows:

- Its underlying graph has n vertices, with adjacency matrix A. That is, $A_{i,j} = 1$ if vertices i and j are connected in the graph and 0 otherwise.
- It has n vertices in each layer, corresponding to the n vertices of the underlying graph.
- Each vertex has n channels.
- The input to each node in the 0-th layer is a one-hot encoding of own identity. That is, the node i in the graph has input $(0, \cdots, 0, \underbrace{1}_{i\text{-th entry}}, 0, \cdots, 0)$.
- The weight connecting node i in layer k to node j in layer $k+1$ is $A_{i,j}$.
- At each layer, the operation at each node is simply to sum up the weighted sum of its inputs and to output the resulting n-dim vector to the next layer. You can think of these as being depth-wise operations if you'd like.

(a) **Write the output of the j-th node at layer k in this network in terms of the matrix A.**
(Hint: This output is an n-dimensional vector since there are n output channels at each layer.)

(b) Recall that a path from i to j in a graph is a sequence of vertices that starts with i, ends with j, and every successive vertex in the sequence is connected by an edge in the graph. The length of a path is the number of edges in it.

Here is some helpful notation:

- $V(i)$ is the set of vertices that are connected to vertex i in the graph.
- $L_k(i,j)$ is the number of distinct paths that go from vertex i to vertex j in the graph where the number of edges traversed in the path is exactly k.
- By convention, there is exactly 1 path of length 0 that starts at each node and ends up at itself. That is, $L_0(i,j) = 1_{i=j}$.

Prove that the i-th output of node j at layer k in the network above is the count of how many paths there are from i to j of length k.

(Hint: Induct on k.)

(c) The GNN we have worked on so far is essentially linear, since the operations performed at each layer are permutation-invariant locally at each node, and can be viewed as essentially doing the exact same thing at each vertex in the graph based on inputs coming from its neighbors. This is called "aggregation" in the language of graph neural nets.

If we represent the graph as a matrix, with the activatios of the i-th node in the i-th row, **what is the update function?**

In the case of the computations in previous parts, **what is the update function that takes the aggregated inputs from neighbors and results in the output for this node?**

(d) The simple GNN described in the previous parts counts paths in the graph. If we were to replace `sum` aggregation with `max` aggregation, **what is the interpretation of the outputs of node j at layer k?**

2. Graph Neural Networks

For an undirected graph with no labels on edges, the function that we compute at each layer of a Graph Neural Network must respect certain properties so that the same function (with weight-sharing) can be used at different nodes in the graph. Let's focus on a single particular "layer" ℓ. For a given node i in the graph, let $\mathbf{s}_i^{\ell-1}$ be the self-message (i.e. the state computed at the previous layer for this node) for this node from the preceeding layer, while the preceeding layer messages from the n_i neighbors of node i are denoted by $\mathbf{m}_{i,j}^{\ell-1}$ where j ranges from 1 to n_i. We will use w with subscripts and superscripts to denote learnable scalar weights. If there's no superscript, the weights are shared across layers. Assume that all dimensions work out.

(a) **Tell which of these are valid functions for this node's computation of the next self-message $\mathbf{s}_i^\ell$. For any choices that are not valid, briefly point out why.**

Note: we are *not* asking you to judge whether these are useful or will have well behaved gradients. Validity means that they respect the invariances and equivariances that we need to be able to deploy as a GNN on an undirected graph.

(i) $\mathbf{s}_i^\ell = w_1 \mathbf{s}_i^{\ell-1} + w_2 \frac{1}{n_i} \sum_{j=1}^{n_i} \mathbf{m}_{i,j}^{\ell-1}$

(ii) $\mathbf{s}_i^\ell = \max(w_1^\ell \mathbf{s}_i^{\ell-1}, w_2 \mathbf{m}_{i,1}^{\ell-1}, w_3 \mathbf{m}_{i,2}^{\ell-1}, \ldots, w_{n_i+1} \mathbf{m}_{i,n_i}^{\ell-1})$ where the $\max$ acts component-wise on the vectors.

(iii) $\mathbf{s}_i^\ell = \max(w_1^\ell \mathbf{s}_i^{\ell-1}, w_2 \mathbf{m}_{i,1}^{\ell-1}, w_2 \mathbf{m}_{i,2}^{\ell-1}, \ldots, w_2 \mathbf{m}_{i,n_i}^{\ell-1})$ where the $\max$ acts component-wise on the vectors.

(b) We are given the following simple graph on which we want to train a GNN. The goal is binary node classification (i.e. classifying the nodes as belonging to type 1 or 0) and we want to hold back nodes 1 and 4 to evaluate performance at the end while using the rest for training. We decide that the surrogate loss to be used for training is the average binary cross-entropy loss.

nodes	1	2	3	4	5
y_i	0	1	1	1	0
$\hat{y}_i$	a	b	c	d	e

Table 1: y_i is the ground truth label, while $\hat{y}_i$ is the predicted probability of node i belonging to class 1 after training.

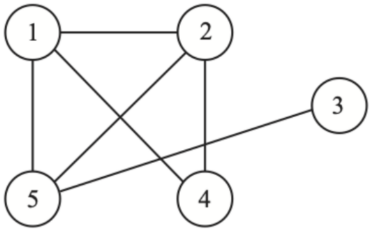


Figure 1: Simple Undirected Graph

Table 1 gives you relevant information about the situation.

Compute the training loss at the end of training.

Remember that with n training points, the formula for average binary cross-entropy loss is

$$\frac{1}{n}\sum_{x}\left(y(x)\log\frac{1}{\hat{y}(x)}+(1-y(x))\log\left(\frac{1}{1-\hat{y}(x)}\right)\right)$$

where the x in the sum ranges over the training points and $\hat{y}(x)$ is the network's predicted probability that the label for point x is 1.

(c) Suppose we decide to use the following update rule for the internal state of the nodes at layer ℓ.

$$\mathbf{s}_i^{\ell}=\mathbf{s}_i^{\ell-1}+W_1\frac{\sum_{j=1}^{n_i}\tanh\left(W_2\mathbf{m}_{i,j}^{\ell-1}\right)}{n_i} \tag{1}$$

where the $\tanh$ nonlinearity acts element-wise.

For a given node i in the graph, let $\mathbf{s}_i^{\ell-1}$ be the self-message for this node from the preceding layer, while the preceding layer messages from the n_i neighbors of node i are denoted by $\mathbf{m}_{i,j}^{\ell-1}$ where j ranges from 1 to n_i. We will use W with subscripts and superscripts to denote learnable weights in matrix form. If there's no superscript, the weights are shared across layers.

(i) **Which of the following design patterns does this update rule have?**

- ☐ Residual connection
- ☐ Batch normalization

(ii) **If the dimension of the state $\mathbf{s}$ is d-dimensional and W_2 has k rows, what are the dimensions of the matrix W_1?**

(iii) If we choose to use the state $\mathbf{s}_i^{\ell-1}$ itself as the message $\mathbf{m}^{\ell-1}$ going to all of node i's neighbors, **please write out the update rules corresponding to (1) giving $\mathbf{s}_i^{\ell}$ for the graph in Figure 1 for nodes $i=2$ and $i=3$ in terms of information from earlier layers.** Expand out all sums.

3. Learning from Point Clouds (Optional)

A point cloud is a discrete *set* of data points in space. Because of this *set*-valued nature of point clouds, concepts from graph neural nets are often relevant in their processing.

In Fig.2, we consider a simple network to process a 2d point cloud $X=\{x_i\}_{i=1}^{n}\in\mathbb{R}^{n\times 2}$, where n is the number of points. The original features of each point are its horizontal and vertical coordinates.

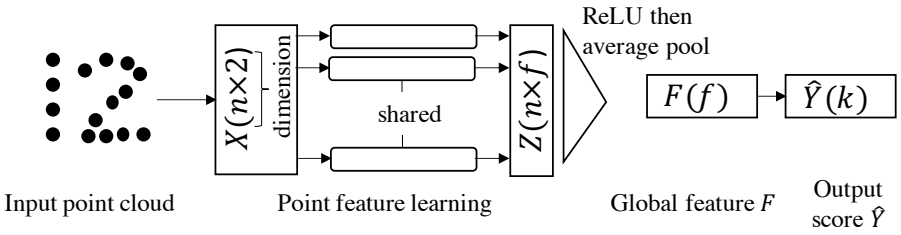


Figure 2: 2d point cloud processing network.

For example, an input point cloud with ground-truth of digit 1 could be represented as $\begin{bmatrix} 0 & 4 \\ 0 & 3 \\ 0 & 2 \\ 0 & 1 \end{bmatrix}$ where each row is a different point in the point cloud.

(a) The *point feature learning* module in Figure 2 learns f-dimensional features for each point separately. Specifically, it learns (shared) weights $W_1 \in \mathbb{R}^{2\times f}$ to get hidden layer outputs $Z = XW_1$. We then apply the nonlinear activation function element-wise and then use average pooling to yield the f-dimensional global feature vector $F \in \mathbb{R}^f$.

Suppose we swap the first two points of the input point cloud X, i.e. $\{x_1, x_2, ..., x_n\}$ to $\{x_2, x_1, ..., x_n\}$. **Show that the global feature F will not change.**

Note: In reality, the network here is *permutation invariant*, as changing the ordering of the n input points in X will not affect the global feature F.

(b) One drawback of the point feature learning discussed in part (a) is that the spatial inter-relationships of different points is not considered.

One idea is to use the Euclidean distance as a similarity measure to group points together into local neighborhoods, and to then process each point together with contextual information about that neighborhood. Your friend proposed several different point grouping methods listed below.

Select all methods that guarantee permutation-invariance, i.e. the global feature vector F will not change with different orderings of the points. Please use ■ for your selections.

☐ For each point x_i of the point cloud X, we find the top-m nearest neighbor points. We then augment each point coordinates with its nearest neighbors' coordinates to make $\widetilde{X} \in \mathbb{R}^{n\times 2(m+1)}$. The order of a concatenated group of points would be: the center point, 1st closest, 2nd closest point, ..., m^{th} closest point. Now $Z = \widetilde{X}\widetilde{W_1}$ where $\widetilde{W_1} \in \mathbb{R}^{2(m+1)\times f}$ are the shared learnable weights.

☐ For each point x_i of the point cloud X, we find the top-m nearest neighbor points. As in the previous choice, we then augment each point coordinates with its nearest neighbors' coordinates and get $\widetilde{X} \in \mathbb{R}^{n\times 2(m+1)}$. *The difference from the previous choice is that the order of a concatenated group of nearby points simply follows their order in the original X.* The $\widetilde{W_1}$ is the same as the previous choice.

☐ For each point x_i of the point cloud X, we *instead find all neighboring points with the distance to x_i smaller than a predefined radius r. We then augment each point coordinates with its neighbors' coordinates with the order being the center point, the 1st closet point within r, the 2nd closet point within r, ..., the furthest point within r.* Because different points might have a different number of neighbors within radius r, the shared learnable weights $\widetilde{W_1} \in \mathbb{R}^{2(n+1)\times f}$ are applied by using the relevant-size truncation of $\widetilde{W_1}$ for every point.

☐ For each point x_i of the point cloud X, as in the previous option, we find all neighboring points with the distance to x_i smaller than a predefined radius r. But instead of concatenating the representation of the point with its neighboring points, we instead extend the point's representation with just the distance d from x_i to the furthest point within the radius r resulting in an $\widetilde{X} \in \mathbb{R}^{n\times 3}$. The $\widetilde{W_1} \in \mathbb{R}^{3\times f}$.

(c) *Point downsampling (reducing the number of points for deeper layers to process).* Let's consider a deeper network, as shown in Fig.3. We are adding a pooling layer (highlighted in bold text) after the first point feature learning layer to downsample half of the points in the cloud ($Z \to Z_1$), followed by another point feature learning layer to increase the dimension of the point features from f to $2f$ ($Z_1 \to Z_2$). (Note that this mimics pooling procedures in CNNs: the spatial size shrinks, followed by an increase of the feature dimensionality.)

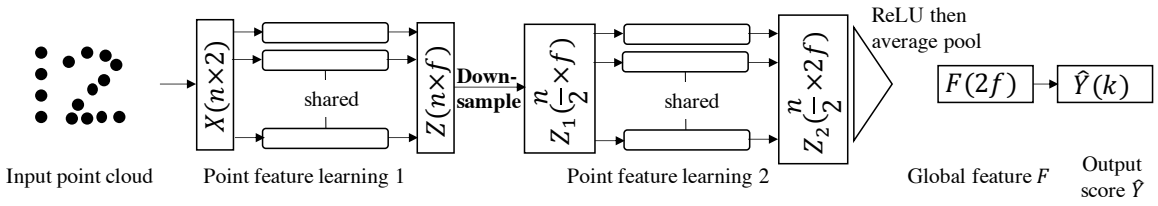


Figure 3: A deeper point cloud processing network.

Consider two candidate algorithms. Both start with the point cloud comprising n points and both iteratively select points until you have at least $\frac{n}{2}$ samples. For both, we construct two sets: **sampled** and **remaining** to denote the set of sampled and remaining points. For both, we first pick a random point and use it to initialize **sampled** and we initialize **remaining** with all the other points. Then, the iterative processes are different for the two algorithms as described below.

Which algorithm is more similar in spirit to using standard max pooling for downsampling in CNNs? Please use ■ for your selections.

☐ **Algorithm 1:**

i. For each point in **remaining** find its nearest neighbor in **sampled**, saving the distance.

ii. Select the point in **remaining** whose nearest neighbor distance is the *largest* and move it from **remaining** to **sampled**. (*i.e.* We keep points far from those we already have.)

☐ **Algorithm 2:**

i. For each point in **remaining** find its nearest neighbor in **sampled**, saving the distance.

ii. Select the point in **remaining** whose nearest neighbor distance is the *smallest* and move it from **remaining** to **sampled**. (*i.e.* We keep points close to those we already have.)

At the end, only the points in **sampled** get sent on to the next layer.

4. Graph Neural Network Conceptual Questions

Diagrams in this question were taken from https://distill.pub/2021/gnn-intro. This blog post is an excellent resource for understanding GNNs and contains interactive diagrams:

A. You are studying how organic molecules break down when heated. For each molecule, you know the element and weight of each atom, which other atoms its connected to, the length of the bond the atoms, and the type of molecule it is (carbohydrate, protein, etc.) You are trying to predict which bond, if any, will break first if the molecule is heated.

(a) How would you represent this as a graph? (What are the nodes, edges, and global state representations? Is it directed or undirected?)

(b) How would you use the outputs of the last GNN layer to make the prediction?

(c) How would you encode the node representation?

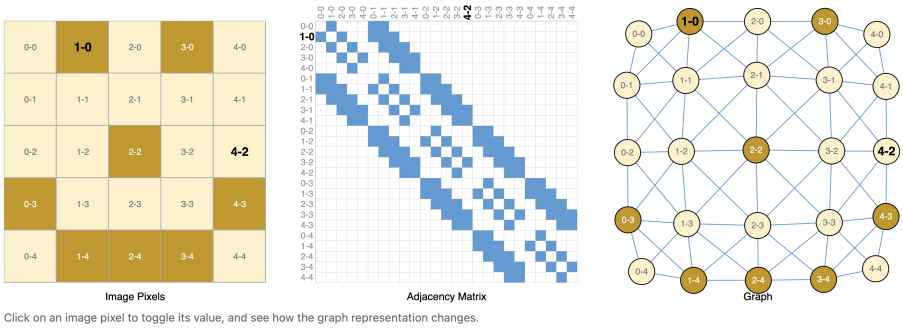


Figure 4: Images as Graphs

B. There are analogs of many ConvNet operations which can be done with GNNs. As Figure 4 illustrates, we can think of pixels as nodes and pixel adjacencies as similar to edges. Graph-level classification tasks, for instance, are analogous to image classification, since both produce a single, global prediction. Fill out the rest of the table. (Not all rows have a perfect answer. The goal is to think about the role an operation serves in one architecture and whether you could use a technique which serves a similar role in the other architecture.)

CNN	GNN
Image classification	Graph-level prediction problem
	Node-level prediction problem
Color jitter data augmentation (adjusting the color or brightness of an image)	
Image flip data augmentation	
Channel dropout	
Zero padding edges	
ResNet skip connections	
Blurring an image	
	Predicting missing values of nodes
	Edge-order invariance - i.e. neighboring nodes/edges are a set with no ordering

C. If you're doing a graph-level classification problem, but node values are missing for some of your graph nodes, how would you use this graph for prediction?

D. Consider the graph neural net architecture shown in Figure 5. It includes representations of nodes (V_n), edges (E_n), and global state (U_n). At each timestep, each node and edge is updated by aggregating neighboring nodes/edges, as well as global state. The global state is the updated by pooling all nodes and edges. For more details on the architecture setup, see https://distill.pub/2021/gnn-intro/#passing-messages-between-parts-of-the-graph.

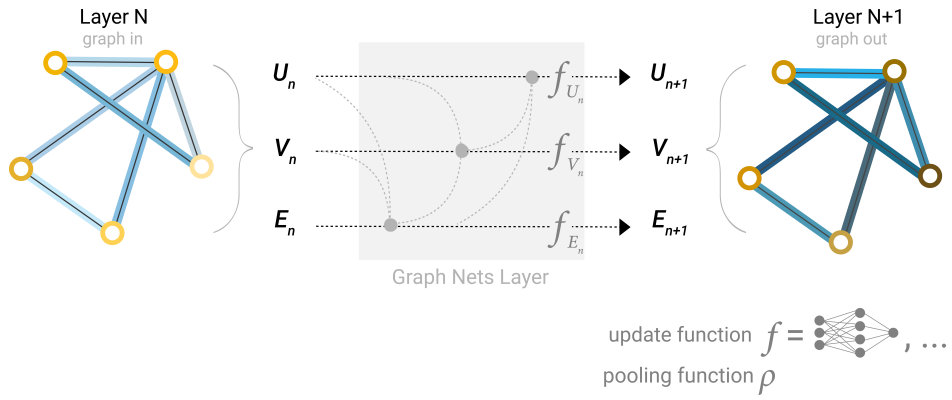


Figure 5: GNN architecture

(a) If we double the number of nodes in a graph which only has node representations, how does this change the number of learned weights in the graph? How does it change the amount of computation used for this graph if the average node degree remains the same? What if the graph if fully connected? (Assume you are not using a global state representation).

(b) Where in this network are learned weights incorporated?

(c) The diagram provided shows undirected edges. How would you incorporate directed edges?

(d) The differences between an MLP and a RNN include (a) weights are shared between timesteps and (b) data is fed in one timestep at a time. How would you make an MLP-like GNN? What about an RNN-like GNN?

E. Let's say you run a large social network and are trying to predict whether individuals in the network like a particular ad. Each individual is represented as a node in the graph, and we are doing a node-level prediction problem. You have labels for around half of the people in the network and are trying to predict the other half. Unlike a traditional ML problem, where there there are many independent training points, here we have a single social network. How would you do prediction on this graph?

F. (Optional) Play around with different GNN design choices in the GNN playground at https://distill.pub/2021/gnn-intro/#gnn-playground. Which design choices lead to the best AUC?

5. Stochastic Gradient Descent (when it is possible to interpolate)

This is a problem about the convergence of SGD for least-squares problems when there is actually a solution that achieves zero loss.

For simplicity, suppose that the problem we are given is

$$X\mathbf{w} = \mathbf{y} \tag{2}$$

where $X = \begin{bmatrix} \mathbf{x}_1^\top \\ \mathbf{x}_2^\top \\ \vdots \\ \mathbf{x}_n^\top \end{bmatrix}$ is a wide matrix with $\mathbf{x}_i$ being d-dimensional vectors and $\mathbf{y} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}$ is an n-dimensional vector. Here, we assume that X has full row-rank (i.e. the $\mathbf{x}_i$ are linearly independent) and so (2) indeed has solutions. (Note that as $d > n$, there there would be infinitely many solutions.)

While we already know lots of ways of solving this problem, it is an illustrative toy example to make sure we understand why SGD works in such settings. This problem has a jupyter demo (**demo link**) attached to it at the end to help you play around with things to get an even deeper set of intuitions.

In this problem, we will just initialize $\mathbf{w}_0 = \mathbf{0}$ for simplicity.

(a) Let's do some preliminaries. First, we want to change coordinates to notationally simplify our analysis of SGD.

Let $\mathbf{w}^*$ be the min-norm solution to (2).

Write out what $\mathbf{w}^*$ is explicitly with respect to X and $\mathbf{y}$ and then, change coordinates to $\mathbf{w}' = \mathbf{w} - \mathbf{w}^*$ to write the new equations as:

$$X\mathbf{w}' = \mathbf{0} \tag{3}$$

What is the new initial condition for $\mathbf{w}'_0$?

(b) Next, let's leverage SVD coordinates to further simplify the problem. **Show that there exists an orthonormal transformation V of variables $\mathbf{w}'' = V\mathbf{w}'$ so that (3) looks like**

$$[\widetilde{X} \;\; \mathbf{0}_{n\times(d-n)}]\mathbf{w}'' = \mathbf{0} \tag{4}$$

and furthermore, **show that the initial condition for $\mathbf{w}'_0$ you computed in the previous part, when viewed as $\mathbf{w}''_0$ has all zeros in the final $(d-n)$ positions.**

(c) **Argue why what you have seen in the previous parts allows us to now focus on a square system of equations:**

$$\widetilde{X}\widetilde{w} = \mathbf{0} \tag{5}$$

and furthermore show that each of the n constituent equations (corresponding to rows) of (5) can be obtained by means of coordinate changes from the same indexed equation in (2).

(d) Let's now engage with SGD itself. Here, we will just use a minibatch length of 1 and batch sampling with replacement. This means that at every iteration of SGD, we roll a fair n-sided die and choose the single equation in (2) that corresponds to the row that came up on the die. Let I_t be the iid uniform random variable on $\{1, \ldots, n\}$ that we roll after iteration t.

At the $t+1$-th iteration, we compute

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \eta \nabla \mathcal{L}_{I_t}(\mathbf{w}_t) \tag{6}$$

where $\mathcal{L}_i(\mathbf{w}) = (y[i] - \mathbf{x}_i^\top \mathbf{w})^2$ is the squared loss on the i-th equation and η is the step-size (learning rate).

Show that an SGD step taken in (6) for the original optimization problem matches exactly to an SGD step taken for $\widetilde{w}$ for solving (5), and that in particular these steps look like:

$$\widetilde{w}_{t+1} = \widetilde{w}_t - 2\eta \widetilde{x}_{I_t} \widetilde{x}_{I_t}^\top \widetilde{w}_t \tag{7}$$

(e) At this point, we can focus entirely on the simplified square system (5) and the stochastic evolution of the iterations described by (7).

To show convergence to zero, we need to pick a suitable stochastic Lyapunov function $\mathcal{L}(\widetilde{w})$ that is bounded below by zero and will decrease in expectation at every time step. In particular, we want to

establish

$$E[\mathcal{L}(\widetilde{w}_{t+1})|\widetilde{w}_t] < (1-\rho)\mathcal{L}(\widetilde{w}_t) \tag{8}$$

with a $1 > \rho > 0$ so that this Lyapunov function tends to decrease exponentially to zero. We will have to have a suitably small step-size/learning-rate η for this to happen, of course.

Show that (8) indeed implies that for every $\epsilon > 0$ and $\delta > 0$, there exists a $T > 0$ for which

$$P(\mathcal{L}(\widetilde{w}_T) < \epsilon) \geq 1 - \delta. \tag{9}$$

(f) One natural guess for a stochastic Lyapunov function is

$$\mathcal{L}(\widetilde{w}) = \widetilde{w}^\top \widetilde{X}^\top \widetilde{X} \widetilde{w}. \tag{10}$$

Argue why the candidate Lyapunov function $\mathcal{L}(\widetilde{w})$ in (10) is non-negative and is only equal to zero at $\widetilde{w} = \mathbf{0}$.

(g) Now, with a guessed stochastic Lyapunov function in hand, we can try to show (8). The first step will be to decompose the evolution of $\mathcal{L}(\widetilde{w})$ into three parts:

$$\mathcal{L}(\widetilde{w}_{t+1}) = \mathcal{L}(\widetilde{w}_t) + A + B \tag{11}$$

where the term A is linear in the actual stochastic update $(\widetilde{w}_{t+1} - \widetilde{w}_t)$ and the term B is quadratic in that update.

Expand out $\mathcal{L}(\widetilde{w}_t + (\widetilde{w}_{t+1} - \widetilde{w}_t))$ to give explicit forms for A and B.

(h) We are counting on the term A in (11) to give us actual contraction in expectation since this looks like a gradient-descent step. **Show:**

$$E[A|\widetilde{w}_t] \leq -c_1 \eta \mathcal{L}(\widetilde{w}_t) \tag{12}$$

where the $c_1 > 0$ is a positive constant that depends on the problem.

(Hint: you are going to want to leverage the actual updates in (7) as well as the singular value structure for $\widetilde{X}$. Can the smallest singular value be zero?)

(i) We need to make sure that the "quadratic" term B in (11) cannot undo the progress made by A in expectation. **Show:**

$$E[B|\widetilde{w}_t] \leq c_2 \eta^2 \mathcal{L}(\widetilde{w}_t) \tag{13}$$

where $c_2 > 0$ is another positive constant that depends on the problem.

(Hint: you are going to want to leverage the actual updates in (7), the singular value structure for $\widetilde{X}$, and the fact that the rows of $\widetilde{X}$ can only be so big. You will want to leverage the largest singular value of $\widetilde{X}$ for one bounding step and then let β be the largest norm of the rows of $\widetilde{X}$ to do another bounding step.)

(j) Finally, we can put the pieces together to see that

$$E[\mathcal{L}(\widetilde{w}_{t+1})|\widetilde{w}_t] \leq (1 - c_1\eta + c_2\eta^2)\mathcal{L}(\widetilde{w}_t) \tag{14}$$

where $c_1 > 0$ and $c_2 > 0$ as well. **Show that this means that there exists a small enough η so that $1 - c_1\eta + c_2\eta^2 < 1$.**

(k) In earlier problem set, you saw how you could reinterpret ridge-regression using feature-aumentation. The earlier parts of this problem have now established that leveraging that trick, you can get SGD to converge exponentially for ridge regression. Check out Jupyter notebook in this **demo link**, and **report what you observed in terms of the convergence rate.**

One of the lessons that you will observe from the code is that the implementation details matter. If you do ridge regression and just treat it as an optimization problem, you won't just be able to use SGD and get exponential convergence with a constant step size. (You would have to adjust the step sizes to make them smaller, but this would slow down your convergence considerably.) But if you intelligently use the feature-aumentation perspective on ridge regression, you'll get exponential convergence.

This is why it is vital for people in EECS to really understand machine learning at the level of detail that we are teaching you. Because in the real world, even if you are a practicing machine learning engineer, if you are working on cutting-edge systems, you need to understand how to implement what you want to do so that it works fast. Equivalent formulations mathematically need not be equivalent from the point of view of implementation – this is one dramatic example of a case when they are not. Take EE227C and beyond if you want to understand these things more deeply.

6. Tensor Rematerialization

You want to train a neural network on a new chip designed at UC Berkeley. Your model is a 10 layer network, where each layer has the same fixed input and output size of s. The chip your model will be trained on is heavily specialized for model evaluation. It can run forward passes through a layer very fast. However, it is severely memory constrained, and can only fit in memory the following items (slightly more than twice of the data necessary for performing a forward pass):

(a) the inputs;

(b) $2s$ activations in memory;

(c) optimizer states necessary for performing the forward pass through the current layer.

To train despite this memory limitation, your friend suggests using a training method called **tensor rematerialization**. She proposes using SGD with a batch size of 1, and only storing the activations of every 5th layer during an initial forward pass to evaluate the model. During backpropagation, she suggests recomputing activations on-the-fly for each layer by loading the relevant last stored activation from memory, and rerunning forward through layers up till the current layer.

Figure 6 illustrates this approach. Activations for Layer 5 and Layer 10 are stored in memory from an initial forward pass through all the layers. Consider when weights in layer 7 are to be updated during backpropagation. To get the activations for layer 7, we would load the activations of layer 5 from memory, and then run them through layer 6 and layer 7 to get the activations for layer 7. These activations can then be used (together with the gradients from upstream) to compute the gradients to update the parameters of Layer 7, as well as get ready to next deal with layer 6.

(a) Assume a forward pass of a single layer is called a `fwd` operation. **How many `fwd` operations are invoked when running a single backward pass through the entire network?** Do not count the initial forward passes required to compute the loss, and don't worry about any extra computation beyond activations to actually backprop gradients.

(b) Assume that each memory access to fetch activations or inputs is called a `loadmem` operation. **How many `loadmem` operations are invoked when running a single backward pass?**

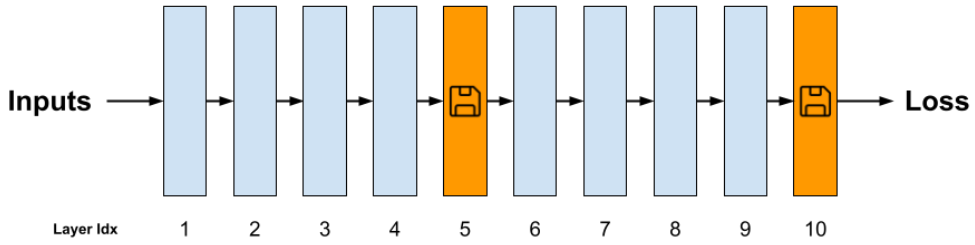


Figure 6: Tensor rematerialization in action - Layer 5 and Layer 10 activations are stored in memory along with the inputs. Activations for other layers are recomputed on-demand from stored activations and inputs.

(c) Say you have access to a local disk which offers practically infinite storage for activations and a `loaddisk` operation for loading activations. You decide to not use tensor rematerialization and instead store all activations on this disk, loading each activation when required. Assuming each `fwd` operation takes 20ns and each `loadmem` operation (which loads from memory, not local disk) takes 10ns for tensor rematerialization, **how fast (in ns) should each `loaddisk` operation be to take the same time for one backward pass as tensor rematerialization?** Assume activations are directly loaded to the processor registers from disk (i.e., they do not have to go to memory first), only one operation can be run at a time, ignore any caching and assume latency of any other related operations is negligible.

7. Backprop through a Simple RNN

Consider the following 1D RNN with no nonlinearities, a 1D hidden state, and 1D inputs u_t at each timestep. (Note: There is only a single parameter w, no bias). This RNN expresses unrolling the following recurrence relation, with hidden state h_t at unrolling step t given by:

$$h_t = w \cdot (u_t + h_{t-1}) \tag{15}$$

The computational graph of unrolling the RNN for three timesteps is shown below:

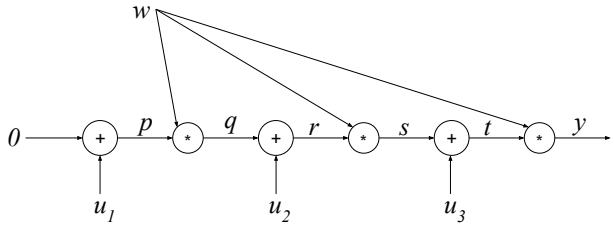


Figure 7: Illustrating the weight-sharing and intermediate results in the RNN.

where w is the learnable weight, u_1, u_2, and u_3 are sequential inputs, and p, q, r, s, and t are intermediate values.

(a) **Fill in the blanks for the intermediate values during the forward pass, in terms of w and the u_i's:**

$$p = u_1 \qquad q = w \cdot u_1 \qquad r = u_2 + q = u_2 + w \cdot u_1$$

$$s = w \cdot r = w \cdot u_2 + w^2 \cdot u_1$$

$$t = \_\_\_\_\_$$

$$y = \_\_\_\_\_$$

(b) **Using the expression for y from the previous subpart, compute $\frac{dy}{dw}$.**

(c) **Fill in the blank for the missing partial derivative of y with respect to the nodes on the backward pass.** You may use values for p, q, r, s, t, y computed in the forward pass and downstream derivatives already computed.

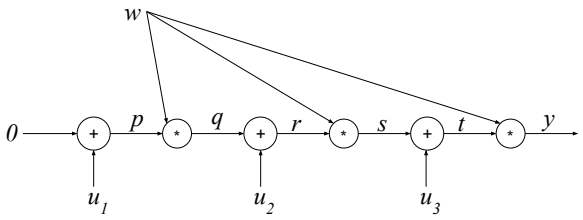
$$\frac{\partial y}{\partial t} = w \qquad \frac{\partial y}{\partial s} = w \qquad \frac{\partial y}{\partial r} = \frac{\partial y}{\partial s} \cdot w \qquad \frac{\partial y}{\partial q} = \frac{\partial y}{\partial r} \cdot 1$$

$$\frac{\partial y}{\partial p} = \_\_\_\_\_$$

(d) **Calculate the partial derivatives along each of the three outgoing edges from the learnable w in Figure 7, replicated below.** (e.g., the right-most edge has a relevant partial derivative of t in terms of how much the output y is effected by a small change in w as it influences y through this edge. You need to compute the partial derivatives for the other two edges yourself.)

You can write your answers in terms of the p, q, r, s, t and the partial derivatives of y with respect to them.

Use these three terms to find the total derivative $\frac{dy}{dw}$.



(HINT: You can use your answer to part (b) to check your work.)

8. Implementing RNNs (and optionally, LSTMs)

This problem involves filling out this notebook.
Note that implementing the LSTM portion of this question is optional and out-of-scope for the exam.

(a) **Implement Section 1A in the notebook**, which constructs a vanilla RNN layer. This layer implements the function

$$h_t = \sigma(W^h h_{t-1} + W^x x_t + b)$$

where W^h, W^x, and b are learned parameter matrices, x is the input sequence, and σ is a nonlinearity such as tanh. The RNN layer "unrolls" across a sequence, passing a hidden state between timesteps and returning an array of hidden states at all timesteps.

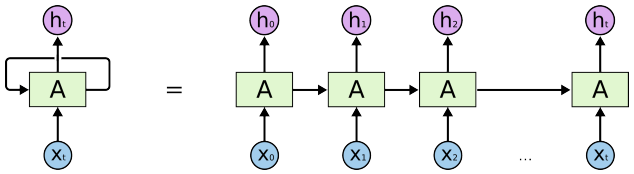


Figure 8: Source: https://colah.github.io/posts/2015-08-Understanding-LSTMs/

Copy the outputs of the *"Test Cases"* code cell and paste it into your submission of the written assignment.

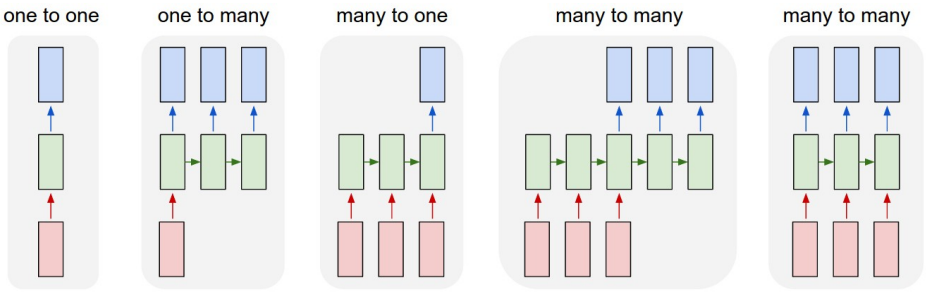
(b) **Implement Section 1.B of the notebook**, in which you'll use this RNN layer in a regression model by adding a final linear layer on top of the RNN outputs.

$$\hat{y}_t = W^f h_t + b^f$$

We'll compute one prediction for each timestep.

Copy the outputs of the *"Tests"* code cell and paste it into your submission of the written assignment.

(c) RNNs can be used for many kinds of prediction problems, as shown below. In this notebook we will look at many-to-one prediction and aligned many-to-many prediction.



We will use a simple averaging task. The input X consists of a sequence of numbers, and the label y is a running average of all numbers seen so far.

We will consider two tasks with this dataset:

- Task 1: predict the running average at all timesteps
- Task 2: predict the average at the last timestep only

Implement Section 1.C in the notebook, in which you'll look at the synthetic dataset shown and implement a loss function for the two problem variants.

Copy the outputs of the *"Tests"* code cell and paste it into your submission of the written assignment.

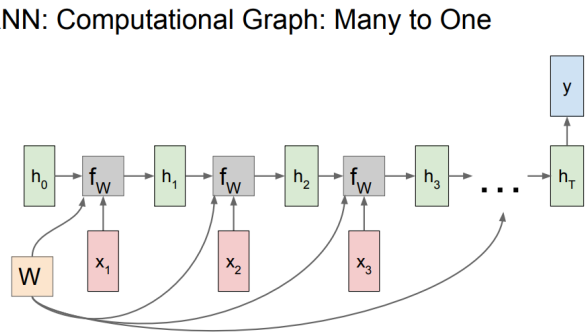


Figure 9: Image source: https://calvinfeng.gitbook.io/machine-learning-notebook/supervised-learning/recurrent-neural-network/recurrent_neural_networks

(d) Consider an RNN which outputs a single prediction at timestep T. As shown in Figure 9, each weight matrix W influences the loss by multiple paths. As a result, the gradient is also summed over multiple paths:

$$\frac{\partial \mathcal{L}}{\partial W} = \frac{\partial \mathcal{L}}{\partial h_T}\frac{\partial h_T}{\partial W} + \frac{\partial \mathcal{L}}{\partial h_{T-1}}\frac{\partial h_{T-1}}{\partial W} + \ldots + \frac{\partial \mathcal{L}}{\partial h_1}\frac{\partial h_1}{\partial W} \tag{16}$$

When you backpropagate a loss through many timesteps, the later terms in this sum often end up with either very small or very large magnitude - called vanishing or exploding gradients respectively. Either problem can make learning with long sequences difficult.

Implement Notebook Section 1.D, which plots the magnitude at each timestep of $\frac{\partial \mathcal{L}}{\partial h_t}$. Play around with this visualization tool and try to generate exploding and vanishing gradients.

Include a screenshot of your visualization in the written assignment submission.

(e) **If the network has no nonlinearities, under what conditions would you expect the exploding or vanishing gradients with for long sequences? Why?** (Hint: it might be helpful to write out the formula for $\frac{\partial \mathcal{L}}{\partial h_t}$ and analyze how this changes with different t). **Do you see this pattern empirically using the visualization tool in Section 1.D in the notebook** with last_step_only=True?

(f) Compare the magnitude of hidden states and gradients when using ReLU and tanh nonlinearities in Section 1.D in the notebook. **Which activation results in more vanishing and exploding gradients? Why?** (This does not have to be a rigorous mathematical explanation.)

(g) **What happens if you set last_target_only = False in Section 1.D in the notebook? Explain why this change affects vanishing gradients. Does it help the network's ability to learn dependencies across long sequences?** (The explanation can be intuitive, not mathematically rigorous.)

(h) (Optional) **Implement Section 1.8 of the notebook** in which you implement a LSTM layer. LSTMs pass a cell state between timesteps as well as a hidden state. **Explore gradient magnitudes using the visualization tool you implemented earlier and report on the results.**

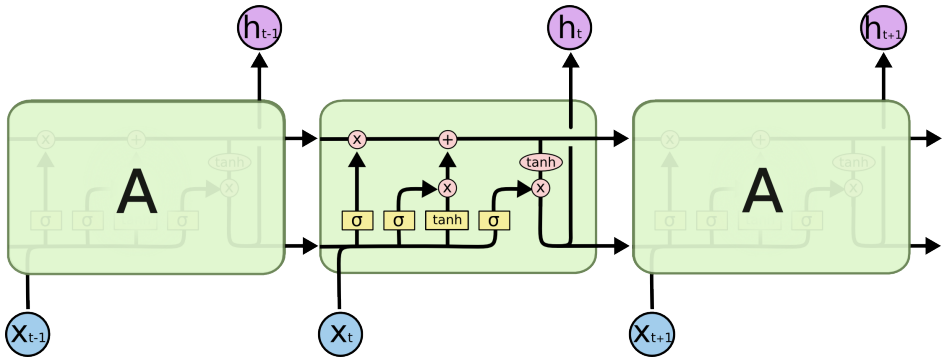


Figure 10: Image source: https://colah.github.io/posts/2015-08-Understanding-LSTMs/

The LSTM forward pass is shown below:

$$
\begin{aligned}
f_t &= \sigma(x_t U^f + h_{t-1} W^f + b^f) \\
i_t &= \sigma(x_t U^i + h_{t-1} W^i + b^i) \\
o_t &= \sigma(x_t U^o + h_{t-1} W^o + b^o) \\
\tilde{C}_t &= \tanh(x_t U^g + h_{t-1} W^g + b^g) \\
C_t &= f_t \circ C_{t-1} + i_t \circ \tilde{C}_t \\
h_t &= \tanh(C_t) \circ o_t
\end{aligned}
$$

where $\circ$ represents the Hadamard Product (elementwise multiplication) and σ is the sigmoid function.

(i) (Optional) When using an LSTM, you should still see vanishing gradients, but the gradients should vanish less quickly. **Interpret why this might happen by considering gradients of the loss with respect to the cell state.** (Hint: consider computing $\frac{\partial \mathcal{L}}{\partial C_{T-1}}$ using the terms $\partial \mathcal{L}, \partial C_T, \partial C_{T-1}, \partial h_T, \partial h_{T-1}$).

(j) (Optional)

Consider a ResNet with simple resblocks defined by $h_{t+1} = \sigma(W_t h_t + b_t) + h_t$. **Draw a connection between the role of a ResNet's skip connections and the LSTM's cell state in facilitating gradient propagation through the network.**

(k) (Optional) We can create multi-layer recurrent networks by stacking layers as shown in Figure 11. The hidden state outputs from one layer become the inputs to the layer above.

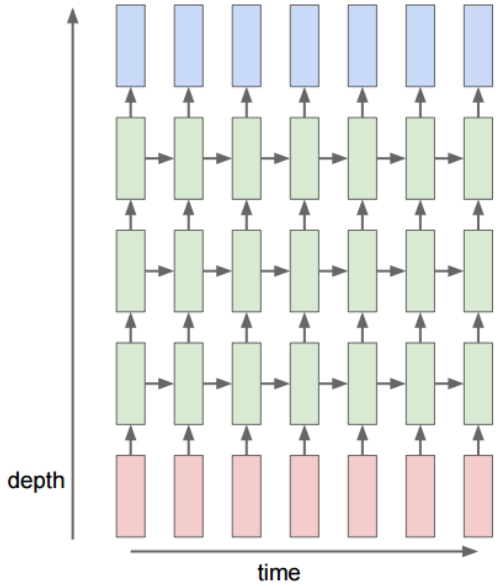


Figure 11: Image source: https://calvinfeng.gitbook.io/machine-learning-notebook/supervised-learning/recurrent-neural-network/recurrent_neural_networks

Implement notebook Section 1.K and run the last cell to train your network. You should be able to reach training loss < 0.001 for the 2-layer networks, and <.01 for the 1-layer networks.

9. Exploring Deep Learning Tooling

Deep learning in practice often requires the use of various tooling in order make the learning workflow efficient. Among these include tools for creating graphs and organizing experiments. These kinds of tools can aid in careful ablation studies, and can accelerate research. We will explore the use of two different tools in this question: tensorboard and wandbai.

(a) Complete the first notebook: tensorboard.ipynb. Provide your graphs here. What is easy about tensorboard? What do you dislike? Would it still be easy to use when we need to run massive amounts of experiments? How organized is it?

(b) Complete the second notebook: wandb.ipynb. No need to provide your graphs. What does wandb have that tensorboard does not?

10. Homework Process and Study Group

Citing sources and collaborators are an important part of life, including being a student!
We also want to understand what resources you find helpful and how much time homework is taking, so we can change things in the future if possible.

(a) **What sources (if any) did you use as you worked through the homework?**

(b) **If you worked with someone on this homework, who did you work with?**
List names and student ID's. (In case of homework party, you can also just describe the group.)

(c) **Roughly how many total hours did you work on this homework?**

Contributors:

- Jerome Quenum.
- Olivia Watkins.
- Anant Sahai.
- Anrui Gu.
- Matthew Lacayo.
- Past EECS 282 and 227 Staff.
- Naman Jain.
- Peter Wang.
- Long He.
- Yaodong Yu.
- Romil Bhardwaj.
- Saagar Sanghavi.
- Dhruv Shah.
- Liam Tan.

